

PROJECT PROPOSAL

Design and Implementation of SecureVault: a Secure Content-Addressed File Storage and Sharing System

Cybersecurity Architecture, Design, and Secure Software Development

Selected Project Option: Option 6 - Secure File Storage System

Prepared by Bernard Appiah, Soumita Bose, Habibullah Turkmen, and Sahil Zainul Aabeddin Kazi

A secure-by-design prototype proposal focused on verifiable storage integrity, authenticated encryption, controlled sharing, and accountable access.

1. Project Title

Design and Implementation of SecureVault: a Secure Content-Addressed File Storage and Sharing System with Authenticated Encryption and Role-Based Access Control.

2. Project Overview

This project proposes a web-based system for securely uploading, storing, sharing, retrieving, and deleting files. Its defining characteristic is a custom-built content-addressed storage engine: every stored file is identified and located by the SHA-256 hash of its content. Identity and integrity therefore coincide; any modification of a stored object changes its hash and is detected on the next read, making the store tamper-evident by construction and deduplicating identical content automatically.

Before reaching disk, every approved file is encrypted with AES-256-GCM under a unique per-file data-encryption key, which is in turn wrapped by a master key supplied through protected configuration. Authentication is implemented from first principles: Argon2id password hashing, opaque server-side session tokens stored only in hashed form, and hardened cookies. Files may be shared with other registered users under owner, editor, and viewer roles, and every file-touching request is authorized through a single centralized access-control function that denies by default. Security-relevant events are recorded in a dedicated audit log, and the entire codebase passes through a continuous-integration pipeline with static analysis, secret scanning, race detection, and dynamic scanning gates.

Committed project boundary: The prototype implements every security mechanism like authentication, sessions, access control, integrity verification, and the storage engine using cryptographic primitives from Go's standard libraries. Off-the-shelf identity providers, object-storage services, and web frameworks are deliberately excluded so that every security-relevant decision is visible in the team's own code and can be explained, tested, and defended. Chunked resumable uploads, public share links, and file version history are excluded from the assessed implementation and retained as planned extensions for the follow-on capstone version.

2.1 Main Objectives

1. Design a layered secure architecture with explicit system components, data flows, protected assets, and four documented trust boundaries.
2. Implement secure registration, login, logout, session handling, and password change following OWASP ASVS and NIST SP 800-63B guidance.
3. Implement a content-addressed storage engine with an atomic write path, hash re-verification on every read, and reference-counted deletion.
4. Encrypt all stored file content with AES-256-GCM using per-file keys, and prevent release of plaintext when authentication, authorization, or integrity verification fails.
5. Implement controlled file sharing between registered users under owner, editor, and viewer roles, enforced by one centralized deny-by-default authorization function.
6. Validate every input on the server: size limits, filename sanitization, MIME and file-signature checks, and strict schema validation of all request fields.
7. Evaluate the prototype through unit tests, an adversarial security test suite, an endpoint-by-role authorization matrix, STRIDE-based threat analysis, and automated static and dynamic scanning.

3. Problem Statement

Multi-user file storage concentrates on the most consequential and most frequently exploited web vulnerability classes in a single system. Broken access control ranked first in the OWASP Top 10 (2021), arises naturally the moment files can be shared: a single missing ownership check turns one user's document into everyone's. Cryptographic failures, ranked second, appear twice: in password storage, where incorrect hashing exposes every account after a database leak, and in data-at-rest protection, where plaintext storage exposes every file after a disk or backup compromise. File-handling flaws such as path traversal and unrestricted upload provide the entry points.

Integrity is the quietest failure mode of all. In a conventional store, silent corruption or deliberate modification of a stored object is undetectable: the system faithfully returns bytes that are no longer the user's file. A trustworthy design must make tampering visible, not merely unlikely.

The guiding question is: *how can a compact academic prototype combine content-addressed integrity, authenticated encryption, from-scratch authentication, and centralized role-based authorization so that the confidentiality, integrity, and authorized availability of stored files are not assumed but demonstrated — each protection paired with an adversarial test that attacks it?*

4. Scope

4.1 Features Included

1. User registration, login, logout, password change, and server-managed authenticated sessions.
2. Two account roles (User and Administrator) plus per-file sharing roles (owner, editor, viewer), with all checks routed through a single centralized authorization function.
3. Upload, authorized listing, download, rename, deletion, and sharing of files with other registered users.
4. A configurable upload size limit enforced while receiving, filename sanitization, and MIME and magic-byte file signature validation.
5. A content-addressed storage engine: SHA-256 content identity, atomic writes via temporary file and rename, hash re-verification on every read, deduplication of identical content, and reference-counted deletion.
6. AES-256-GCM encryption of all stored content with a unique per-file data-encryption key wrapped by a master key from protected configuration.
7. Audit logging of authentication events, authorization denials, uploads, downloads, deletions, sharing changes, and integrity failures.
8. A minimal web interface (login, file list, upload with progress, download, and a sharing dialog) operating under a strict Content-Security-Policy.
9. A continuous-integration pipeline with static analysis (Semgrep), secret scanning (gitleaks), race detection, and an OWASP ZAP baseline scan before submission.

4.2 Explicit Exclusions

1. Chunked or resumable uploads, delta synchronization, and desktop synchronization clients

2. Public download links, share-link passwords or expiry, and sharing with unregistered recipients.
3. File version history, full-text search, thumbnails, and background job processing.
4. Antivirus or content-analysis scanning of uploaded files; uploads are constrained by validation and size policy rather than content inspection.
5. Enterprise high availability, distributed or multi-node storage, cloud key-management services, and large-scale performance testing.
6. A mobile application and real-time collaboration.

4.3 Sharing Roles and Permitted Actions

Role	Granted by	Permitted actions
Owner	Uploading the file	View metadata, download, rename, delete, grant and revoke editor or viewer roles.
Editor	Owner	View metadata, download, and rename; may not delete or re-share.
Viewer	Owner	View metadata and download only.
Administrator	System configuration	Account administration and audit review; administrative actions are audited and cannot read file plaintext through the sharing model.
No grant	—	Every request is denied by default and the denial is recorded in the audit log.

5. Project Approach

5.1 Architecture and Trust Boundaries

The prototype uses a layered architecture separating the untrusted client, the TLS entry point, the trusted application zone, and the protected data zone. The browser never receives a filesystem path and never accesses storage directly: the Go application mediates every operation, authentication is verified at the session layer, and authorization is evaluated again at the point of every file access.

Ciphertext objects, relational metadata, and the audit log reside in the protected data zone and are reachable only through the storage engine and the database driver.

SecureVault — High-Level Logical Architecture

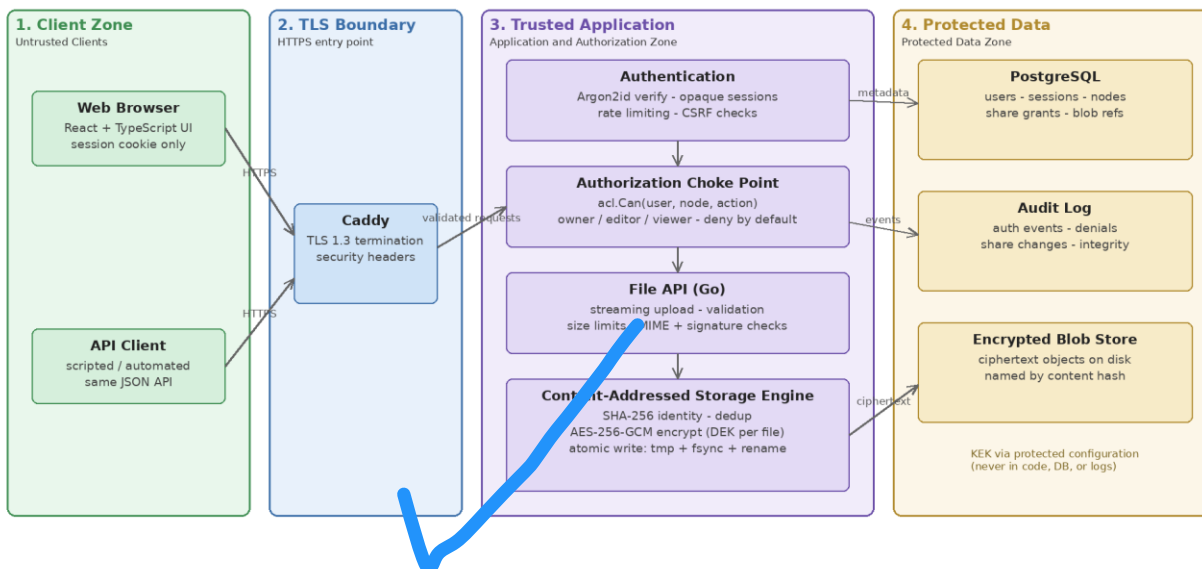


Figure 1. Proposed logical architecture, principal data flows, and trust boundaries.

5.2 Secure File Workflow

1. **Authenticate and authorize.** The system verifies the session token against its hashed record and confirms the account may upload.
2. **Receive safely.** The application streams the upload with a hard size limit enforced while receiving, sanitizes the original filename into escaped metadata, and never constructs a filesystem path from user input.
3. **Validate.** The declared MIME type, detected magic-byte signature, and size policy are checked on the server; any mismatch rejects the upload.
4. **Fingerprint.** The exact received bytes are hashed with SHA-256 while streaming; this fingerprint becomes the object's permanent identity and integrity reference.
5. **Protect.** The system generates a fresh data-encryption key and nonce, encrypts the content with AES-256-GCM, wraps the key with the master key, and writes the ciphertext atomically (temporary file, fsync, rename) under a name derived from the content fingerprint.

6. Record. File metadata, ownership, and the wrapped key are stored in PostgreSQL in the same transaction that makes the file visible; an audit event is written.
7. Retrieve securely. The application re-checks the session, evaluates the sharing role through the central authorization function, verifies the AES-GCM authentication tag during decryption, re-verifies the plaintext SHA-256 against the recorded fingerprint, and only then streams the file as an attachment.
8. Delete and audit. Authorized deletion decrements the object's reference count, removes it when unreferenced, and preserves a minimal, non-sensitive audit event.

5.3 Security Implementation Plan

Security control	Planned implementation
Authentication and sessions	Argon2id hashes with calibrated parameters and per-password salts; uniform login errors with timing equalization; opaque random session tokens stored SHA-256-hashed; HttpOnly, Secure, SameSite cookies; session rotation on login; CSRF protection; login throttling.
Authorization	Ownership, account-role, and share-grant (owner/editor/viewer) checks on every list, download, rename, share, delete, and administrative action, all evaluated through one centralized deny-by-default function.
Input and upload validation	Strict server-side schema validation of all request fields; streaming size enforcement; filename sanitization; MIME and magic-byte signature checks; UUID-free content-hash storage names; no user-controlled paths.
Storage integrity	SHA-256 content addressing; atomic temporary-file-and-rename writes; hash re-verification on every read; reference-counted deletion; corruption reported, logged, and never served.

Encryption and key handling	AES-256-GCM with a fresh 96-bit nonce per file; per-file data-encryption keys wrapped by a master key supplied through protected configuration and excluded from source control, logs, and the database.
Logging and monitoring	Timestamped audit events with actor, action, target, result, and reason code; passwords, keys, tokens, and file contents are never logged; structured application logs with request identifiers.
Secure communication and headers	TLS 1.3 via a Caddy reverse proxy; HSTS, Content-Security-Policy, and X-Content-Type-Options headers; attachment-only download responses; parameterized database access exclusively.
Secure delivery pipeline	Git workflow with mandatory review; continuous integration running vet, static analysis (Semgrep), secret scanning (gitleaks), and race-detected tests on every commit; OWASP ZAP baseline scan with all medium-or-higher findings resolved.

5.4 Implementation Phases

1. Requirements and threat modelling: finalize assets, abuse cases, trust boundaries, database schema, and acceptance tests.
2. Storage engine: implement and unit-test the content-addressed store with hashing, atomic writes, encryption, and reference counting as a standalone library before any web code.
3. Authentication: registration, login, sessions, throttling, and the adversarial authentication test suite.
4. File API and sharing: upload pipeline with validation, download path, sharing model, centralized authorization, and audit logging.
5. Hardening and validation: web interface, security headers, the endpoint-by-role authorization matrix, Semgrep and gitleaks gates, and OWASP ZAP testing.
6. Evidence and presentation: capture test evidence, complete the report and diagrams, rehearse the demonstration, and prepare question-and-answer responses.

6. Preliminary Threat Model and Risk Analysis

6.1 Threat Analysis

STRIDE will be applied to the data-flow diagram at each external interface, process, data store, and trust-boundary crossing. Likelihood and impact will be rated Low, Medium, or High, and each identified threat will be linked to an implemented control and to the specific test that verifies the control. Particular attention is given to broken object-level authorization (the dominant risk of any sharing system), to integrity of stored content, and to injection and improper-input handling across authentication forms, route parameters, and file metadata.

6.2 Preliminary Risk Register

Category	Threat scenario	Risk	Controls and verification
Spoofing	Credential guessing or session theft allows an attacker to act as another user.	High	Argon2id, uniform errors with timing equalization, throttling, hashed session tokens, secure cookies, rotation. Verify with brute-force, timing, and stolen or expired session tests.
Tampering	An attacker or fault modifies ciphertext on disk or metadata in the database.	High	AES-GCM tag verification, SHA-256 re-verification against the content-addressed identity, restricted permissions. Flip ciphertext bits and confirm retrieval is blocked and logged.
Repudiation	A user denies uploading, sharing, downloading, or deleting a file.	Medium	Audit events with actor, target, time, outcome, and reason for successful and denied operations. Verify event creation across the lifecycle.

Information disclosure	A user manipulates identifiers or abuses a sharing grant to read another user's file, or reads plaintext from disk.	High	Centralized owner/role/grant checks, encryption at rest, attachment-only responses. Run cross-user and cross-role (no grant, viewer, editor) access tests and inspect raw storage.
Denial of service	Oversized or repeated uploads exhaust disk, memory, or connections.	Medium	Streaming size limits enforced while receiving, request throttling, quotas, safe cleanup of temporary files. Test oversized and interrupted uploads.
Elevation of privilege	A normal user invokes administrative functions or escalates a viewer grant to editor rights.	High	Deny-by-default checks on every administrative and sharing endpoint. Invoke each endpoint with insufficient roles and expect denial and an audit event.
Path traversal / stored XSS	A crafted filename influences a filesystem path or renders active content in the interface.	High	No user-controlled paths (content-hash storage names), filename escaping, output encoding, strict Content-Security-Policy, safe Content-Disposition. Test traversal strings and HTML-like filenames.
SQL injection	Crafted input through authentication fields, identifiers, or metadata manipulates a database query.	High	Parameterized queries exclusively via the pgx driver; strict schema validation; least-privilege database account. Verify with injection payloads and OWASP ZAP.

Improper / unvalidated input	Malformed, overlong, or inconsistently typed values bypass policy checks or trigger unsafe behaviour.	High	Explicit server-side schemas, allowlists, type and length limits, fail-closed handling without stack traces. Verify with malformed identifiers, boundary values, and unusual encodings.
-------------------------------------	---	------	---

7. Testing and Validation Plan

Testing combines unit tests of the storage engine, an adversarial security suite, an endpoint-by-role authorization matrix, race-detected concurrency tests, static analysis, and a baseline dynamic scan. Every claimed protection is paired with a test that attacks it.

Scenario	Controlled action	Expected secure result
Valid lifecycle	Upload an allowed clean file, then list, download, share, and delete it as its owner.	Stored object is ciphertext; downloaded bytes match the original SHA-256; events are logged.
Deduplication	Upload the same file twice from two accounts.	One stored object with two references; deletion by one account does not affect the other.
Authentication	Submit invalid credentials repeatedly; access a protected route without a session; measure response timing for known and unknown accounts.	Uniform failure messages, throttling after the threshold, no protected response, no measurable timing difference.
Session handling	Inspect cookie attributes; reuse a token after logout and after rotation.	HttpOnly, Secure, SameSite set; invalidated tokens rejected; database compromise reveals only hashed tokens.

Authorization matrix	Exercise every endpoint as owner, editor, viewer, unrelated user, administrator, and unauthenticated client.	Every combination returns exactly the permitted outcome; all denials are recorded.
Object-level authorization	User B requests User A's file for metadata, download, and deletion with no grant, a viewer grant, and an editor grant.	Insufficient grants are denied and recorded; a viewer grant permits download only; no unauthorized bytes or metadata are disclosed.
Filename / path handling	Upload files whose names contain traversal sequences, reserved names, or HTML-like characters.	No user-controlled path is created; the displayed name is escaped; storage uses the content hash.
Type spoofing	Rename a disallowed file to an allowlisted extension and upload it.	Signature and MIME mismatch is detected and the upload is rejected.
Oversized upload	Submit a file exceeding the configured limit.	The stream is stopped at the limit, no object is created, and temporary data is removed.
Integrity attack	Modify bytes of a stored ciphertext object, then request the file.	AES-GCM verification fails, plaintext is not released, the hash mismatch is logged, and the client receives a controlled error.
Crash consistency	Terminate the server mid-write repeatedly under load.	No partially written object ever becomes visible; the store passes verification after restart.
Application security	Run Semgrep, gitleaks, race-detected tests, and an OWASP ZAP baseline scan.	No unresolved high-severity finding; results and justified exceptions are documented.

8. Expected System Delivery

1. A working secure file-storage and sharing web prototype and complete source repository with continuous-integration history.
2. A logical architecture diagram, data-flow representation, and clearly marked trust boundaries.
3. Database design and a documented secure upload, sharing, and retrieval workflow.
4. Implemented authentication, authorization, validation, content-addressed integrity, encryption, and audit features.
5. A STRIDE threat model, risk register, mitigations, and traceability from threats to tests.
6. Automated functional and security test cases, the authorization matrix, scan reports, and attack-demonstration evidence.
7. A final report following the required structure, including challenges and lessons learned.

9. Technology Stack

Component	Selected technology
Language and framework	Go 1.22+ (standard net/http; single static binary)
Frontend	React with TypeScript (Vite), under a strict Content-Security-Policy
Database	PostgreSQL 16 with the pgx driver; parameterized queries only
Authentication	Argon2id through golang.org/x/crypto/argon2 ; server-managed opaque sessions
Encryption and integrity	Go standard cryptography libraries; AES-256-GCM; SHA-256 content fingerprints
File identification	Magic-byte/file-signature detection in Go plus MIME validation

Testing	Go test with race detection, httptest, Semgrep, gitleaks, and OWASP ZAP
Deployment and reproducibility	Docker Compose for the application, PostgreSQL, and Caddy (TLS); Git and GitHub with CI security gates on every commit

10. Team Roles and Responsibilities

All members contribute to design, implementation, testing, and documentation. The assignments below denote lead responsibility; weekly integration meetings and mandatory cross-review of pull requests ensure that every member can explain every component during the presentation and question session.

Habibullah Turkmen: Architecture, content-addressed storage engine, and the CI security pipeline; technical review of all components.

Bernard Appiah: authentication and sessions: registration and login flows, Argon2id integration, session lifecycle, throttling, and the adversarial authentication test suite.

Soumita Bose: file API, sharing model, and centralized authorization; the endpoint-by-role authorization matrix and audit logging.

Sahil Zainul Aabeddin Kazi: web interface, upload validation, and testing and validation evidence; assembly of the report, diagrams, and presentation materials.

11. Feasibility and Risk-Control Strategy

The scope deliberately concentrates on depth over breadth. A single Go binary, one database, and a minimal interface remove framework and infrastructure complexity; the storage engine is built and fully tested as a standalone library before any web code depends on it, so the highest-risk

component is de-risked first. Docker Compose provides a repeatable environment identical between development, testing, and the live demonstration.

The core acceptance path — storage engine, authentication, file API with sharing, and the security test suite — will be completed before optional refinements. Interface polish, additional file-type policies, or richer metrics may be added only if they do not delay the required system, threat model, tests, report, and demonstration.

12. Conclusion

The proposed system provides a realistic, security-focused implementation of the Secure File Storage System option. Its distinguishing property is that integrity is structural rather than assumed: content-addressed storage makes every stored object self-verifying, authenticated encryption protects confidentiality at rest, from-scratch authentication and centralized role-based authorization control every access, and a dedicated audit trail makes actions accountable. The scope is sufficiently substantial to demonstrate secure architecture and attack prevention, yet constrained enough for a four-member team to implement, test, document, and demonstrate reliably, with each member holding lead responsibility for one major component.

By pairing each major threat with a control and a controlled verification scenario — including bit-level tamper tests, crash-consistency tests, and a complete endpoint-by-role authorization matrix — the project demonstrates secure-by-design thinking rather than presenting security as an afterthought. The prototype further constitutes version 0.1 of a longer-term system that the team intends to extend as a Master's capstone project and subsequently release as open source; the boundary above is scoped to what will be demonstrably complete by the course deadline.

References

OWASP Foundation. Application Security Verification Standard (ASVS).

<https://owasp.org/www-project-application-security-verification-standard/>

OWASP Foundation. Password Storage Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. Session Management Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP Foundation. Cryptographic Storage Cheat Sheet.

https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

OWASP Foundation. OWASP Top 10:2021. <https://owasp.org/Top10/>

National Institute of Standards and Technology. NIST SP 800-63B: Authentication and Authenticator Management. <https://pages.nist.gov/800-63-4/sp800-63b.html>

Microsoft. Security Threat Models and the STRIDE Approach. <https://learn.microsoft.com/en-us/windows-hardware/drivers/ifs/designing-with-security-threat-models>

The Go Authors. Package documentation: crypto/cipher, crypto/sha256, and golang.org/x/crypto/argon2. <https://pkg.go.dev/>